NS-3 NETWORK SIMULATION PROJECT

Comparative Performance Evaluation of DropTail, RED and CoDel

Under Variable Network Congestion Using NS-3

ACTIVE QUEUE MANAGEMENT

SUBMISSION DETAILS	
Submitted by	Anand Kumar
Roll number	
Course	Data Communication and Networking Lab
Department	Electronics and Communication Engineering
Submitted to	Dr. Monica Bhutani

Simulation Platform: NS-3

Abstract
Network congestion is not determined by link capacity alone. The way a router manages packets waiting for transmission can change the delay experienced by applications, the amount of data successfully delivered, and the extent to which competing flows share a constrained link. This project studies that behaviour through a controlled NS-3 simulation.

A two-router dumbbell topology is configured with concurrent TCP and UDP traffic. The queue discipline on the shared 10 Mbps bottleneck is changed between DropTail, Random Early Detection (RED), and Controlled Delay (CoDel). For each discipline, the UDP offered load is set to 4 Mbps, 8 Mbps, and 12 Mbps, giving nine experimental cases. TCP throughput, UDP throughput, average FlowMonitor delay, average FlowMonitor jitter, and FlowMonitor lost-packet count are recorded.

The results show a consistent deterioration in TCP throughput as UDP load increases. At 12 Mbps of offered UDP traffic, TCP throughput falls to 0.041 Mbps with DropTail, 0.145 Mbps with RED, and 0.105 Mbps with CoDel. RED records the lowest aggregated delay in all three tested load conditions. CoDel has the smallest FlowMonitor loss count at the lowest UDP load, while DropTail records the highest delivered UDP throughput in each case. These outcomes point to trade-offs rather than a universally superior queue discipline.

The report also distinguishes measured FlowMonitor statistics from direct queue-disc measurements. This distinction matters: the experiment aggregates delay and jitter over monitored flows, and the recorded lost-packet count is not a direct count of packets dropped by the bottleneck queue.

Keywords
NS-3, Active Queue Management, DropTail, Random Early Detection, CoDel, dumbbell topology, TCP, UDP, congestion, FlowMonitor.

1. Introduction

When packets reach a router faster than its outgoing interface can transmit them, the excess packets wait in a buffer. A buffer is necessary because traffic is rarely perfectly uniform. However, a queue that remains full for long periods can become a source of substantial delay. Applications may continue receiving data, yet the packets can spend an unnecessarily long time waiting before transmission. This condition is commonly associated with bufferbloat.

Queue management determines what happens while this congestion develops. A simple queue may accept packets until its capacity is exhausted and then discard new arrivals. More advanced approaches attempt to respond before the buffer remains persistently full. Active Queue Management (AQM) algorithms are designed to provide this earlier congestion signal.

This practical compares three approaches. DropTail represents the conventional finite FIFO queue. RED estimates average queue occupancy and uses probabilistic early marking or dropping. CoDel takes a different route: it observes how long packets remain in the queue and responds when that delay stays above a configured target for a sufficient interval.

The comparison is conducted under mixed traffic rather than a single isolated flow. A TCP source and a constant-rate UDP source share the same bottleneck. Increasing the UDP offered load makes it possible to observe how each queue discipline behaves as the link approaches and exceeds its available service capacity.

2. Aim and Objectives

To evaluate and compare DropTail, RED, and CoDel under increasing UDP offered load in a mixed TCP/UDP dumbbell network simulated using NS-3.

- Construct a dumbbell topology with two routers connected by a shared bottleneck link.
- Generate concurrent TCP BulkSend and constant-rate UDP traffic.
- Evaluate each queue discipline at UDP offered loads of 4 Mbps, 8 Mbps, and 12 Mbps.
- Measure TCP and UDP application throughput, aggregated FlowMonitor delay and jitter, and FlowMonitor lost-packet count.
- Compare the observed trade-offs and identify what the collected statistics can—and cannot—establish.

3. Technical Background

3.1 DropTail

DropTail is a straightforward FIFO queue discipline. Incoming packets are placed at the end of the queue while buffer space is available. Once the configured limit is reached, newly arriving packets are discarded. Its main advantage is simplicity; there is little queue-management logic to configure.

The limitation becomes visible under sustained congestion. DropTail does not deliberately signal congestion while the queue is still building. The queue can therefore grow towards its limit, increasing packet waiting time. When the buffer finally fills, packet loss may occur in bursts.

3.2 Random Early Detection (RED)

RED monitors an estimate of the average queue size rather than reacting only to the instantaneous queue limit. When this average moves into a configured congestion region, RED begins marking or dropping packets with a probability that increases with the average queue occupancy. The intention is to provide an earlier congestion signal and reduce the likelihood of a persistently full queue.

RED is parameter-sensitive. Its behaviour depends on threshold values, maximum marking or dropping probability, queue limit, packet-size assumptions, and averaging configuration. In this implementation, the source explicitly sets the minimum and maximum thresholds, the maximum queue size, link bandwidth, link delay, and mean packet size.

3.3 Controlled Delay (CoDel)

CoDel is based on packet sojourn time: the time a packet spends waiting in the queue. Rather than relying only on queue length, it looks for persistent excessive delay. If the minimum sojourn time remains above the target for the configured interval, CoDel begins dropping or marking packets to signal congestion.

This delay-oriented approach is intended to control standing queues while allowing short bursts to pass. The supplied program sets CoDel's maximum queue size to 100 packets but does not explicitly override its target or interval attributes. Those attributes therefore remain at the defaults of the NS-3 CoDel model used for the run; the exact NS-3 release should be recorded when reproducing the experiment.

4. Network Topology and Configuration

The simulation uses a dumbbell arrangement. Two client nodes connect to Router 1, which forwards traffic across a shared point-to-point bottleneck to Router 2. On the destination side, Router 2 connects to separate TCP and UDP server nodes. The TCP and UDP paths therefore compete for the same inter-router link.

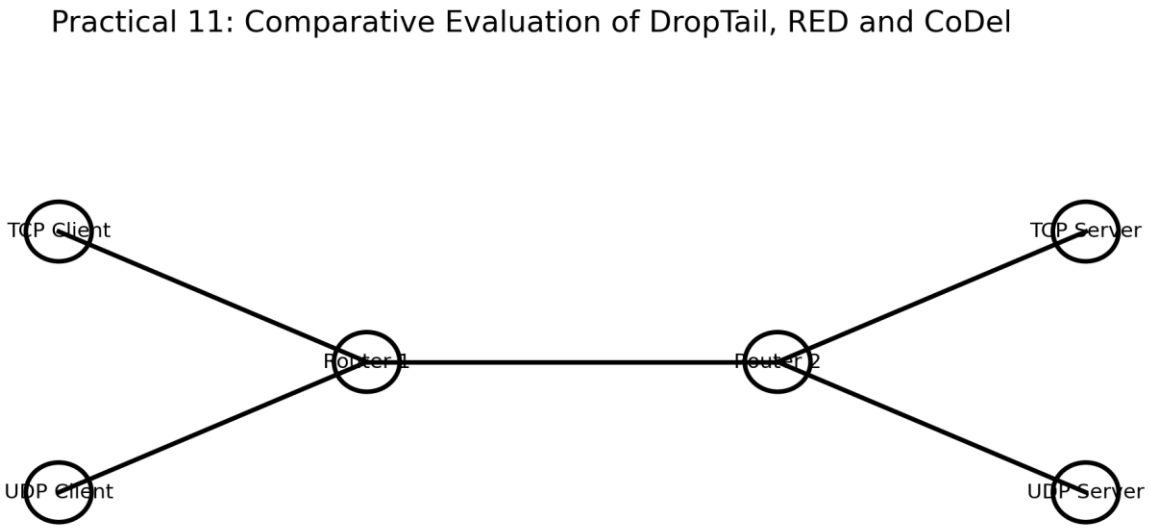


Figure 1. Network topology used for Practical 11. The shared link between Router 1 and Router 2 is the congestion bottleneck.

The access links are configured at 100 Mbps with a 2 ms channel delay. The inter-router bottleneck is configured at 10 Mbps with a 10 ms channel delay. A 100-packet maximum queue size is used for each tested root queue discipline.

Parameter	**Value used in the supplied source code**
Simulator	NS-3
Topology	Two clients – Router 1 – Router 2 – two servers
Number of nodes	6 (2 clients, 2 routers, 2 servers)
Access-link data rate	100 Mbps
Access-link channel delay	2 ms
Bottleneck data rate	10 Mbps
Bottleneck channel delay	10 ms
Queue maximum size	100 packets
TCP application	BulkSend over TCP; MaxBytes = 0 (unlimited)
UDP application	OnOff application with constant On time and zero Off time
UDP packet size	1024 bytes
UDP offered rates	4 Mbps, 8 Mbps, 12 Mbps
Simulation stop time	20 seconds
Application transmission interval	1 s to 19 s (18 seconds)
Statistics	PacketSink counters and FlowMonitor

4.1 IP addressing and routing

Separate /24 subnets are assigned to the two client access links, the inter-router link, and the two server access links. The source code uses the network prefixes 10.1.1.0/24, 10.1.2.0/24, 10.1.3.0/24, 10.1.4.0/24, and 10.1.5.0/24 respectively. Global routing tables are populated after the addresses are assigned.

4.2 Queue-discipline configuration

Queue discipline	Configuration in mini-project11.cc
DropTail	ns3::FifoQueueDisc; MaxSize = 100p
RED	ns3::RedQueueDisc; MinTh = 5; MaxTh = 15; MaxSize = 100p; LinkBandwidth = 10Mbps; LinkDelay = 10ms; MeanPktSize = 1024 bytes
CoDel	ns3::CoDelQueueDisc; MaxSize = 100p; target and interval not explicitly overridden

For every scenario, the selected queue discipline is installed on the bottleneck devices after the existing root queue discipline is uninstalled. The remaining topology and traffic settings are kept the same while the algorithm and UDP offered rate are changed.

5. Experimental Methodology

The experiment is designed as a nine-case comparison. Each queue discipline is tested at all three UDP offered rates. Only the queue discipline and the UDP source rate vary between cases; the topology, link properties, packet size, application timing, and simulation duration remain fixed.

1. Create two client nodes, two router nodes, and two server nodes.
2. Connect each client to Router 1 and Router 2 to each server using 100 Mbps point-to-point access links with 2 ms delay.
3. Connect Router 1 and Router 2 using a 10 Mbps point-to-point bottleneck with 10 ms delay.
4. Install the NS-3 Internet stack, assign the five IP subnets, and populate global routing tables.
5. Install one queue discipline on the bottleneck for the current run: DropTail, RED, or CoDel.
6. Start the TCP receiver and UDP receiver at 0 seconds; start both senders at 1 second and stop them at 19 seconds.
7. Configure TCP BulkSend with unlimited MaxBytes. Configure the UDP OnOff source to transmit continuously using 1024-byte packets.
8. Run the scenario once at each UDP offered rate: 4 Mbps, 8 Mbps, and 12 Mbps.
9. Use PacketSink byte counters to calculate application throughput and FlowMonitor to collect delay, jitter, and lost-packet statistics.
10. Repeat the procedure for all three queue disciplines, producing nine sets of measurements.

5.1 Throughput calculation

The source code calculates application throughput from the total number of bytes received by each PacketSink. The active sending interval is 18 seconds, from 1 s to 19 s. The calculation used for both TCP and UDP is:

Throughput (Mbps) = Total received bytes × 8 / (18 × 10^6)

This is an application-level received-data rate based on PacketSink byte counters. It should not be confused with the nominal link data rate or with a count of all protocol overhead transmitted on the network.

5.2 FlowMonitor calculations

The program obtains the FlowMonitor statistics for all monitored flows. It adds the delay sums and jitter sums, adds the received-packet counts, and then divides the accumulated delay and jitter by the total number of received packets. Lost-packet counts are also summed across the monitored flows.

Average delay = Σ(flow delaySum) / Σ(flow rxPackets)

Average jitter = Σ(flow jitterSum) / Σ(flow rxPackets)

The resulting delay and jitter values are therefore aggregated statistics. They are not isolated measurements for the TCP application or UDP application alone.

6. Performance Metrics

Metric	What it represents	How it is interpreted here
TCP throughput	TCP payload bytes received per second at the TCP PacketSink.	Indicates how much application data the TCP flow delivers while competing with UDP.
UDP throughput	UDP payload bytes received per second at the UDP PacketSink.	Shows the delivered UDP rate compared with the configured offered rate.
Average FlowMonitor delay	Total recorded delay divided by total received packets across monitored flows.	Provides an aggregate indication of packet delivery delay; it is not application-specific.
Average FlowMonitor jitter	Total recorded jitter divided by total received packets across monitored flows.	Shows aggregate packet-delay variation, subject to the mix of monitored flows.
FlowMonitor lost packets	Sum of lostPackets reported by FlowMonitor across flows.	A FlowMonitor loss statistic, not a direct bottleneck queue-drop counter.

7. Simulation Results

The following values are taken from the supplied experiment output and CSV file. Throughput is expressed in Mbps, delay and jitter in milliseconds, and lost packets as a count. No additional simulation runs have been introduced.

Queue discipline	UDP load (Mbps)	TCP throughput (Mbps)	UDP throughput (Mbps)	Avg. delay (ms)	Avg. jitter (ms)	Lost packets
DropTail	4	5.392	3.998	64.565	0.369	51
DropTail	8	1.725	7.971	111.659	0.318	215
DropTail	12	0.041	9.756	179.381	0.251	5016
RED	4	5.087	3.990	45.822	0.357	65
RED	8	1.707	7.906	66.312	0.316	386
RED	12	0.145	9.676	111.343	0.269	5174
CoDel	4	5.385	3.997	46.669	0.358	13
CoDel	8	1.677	7.956	67.409	0.312	223
CoDel	12	0.105	9.740	156.298	0.269	5041

7.1 TCP throughput

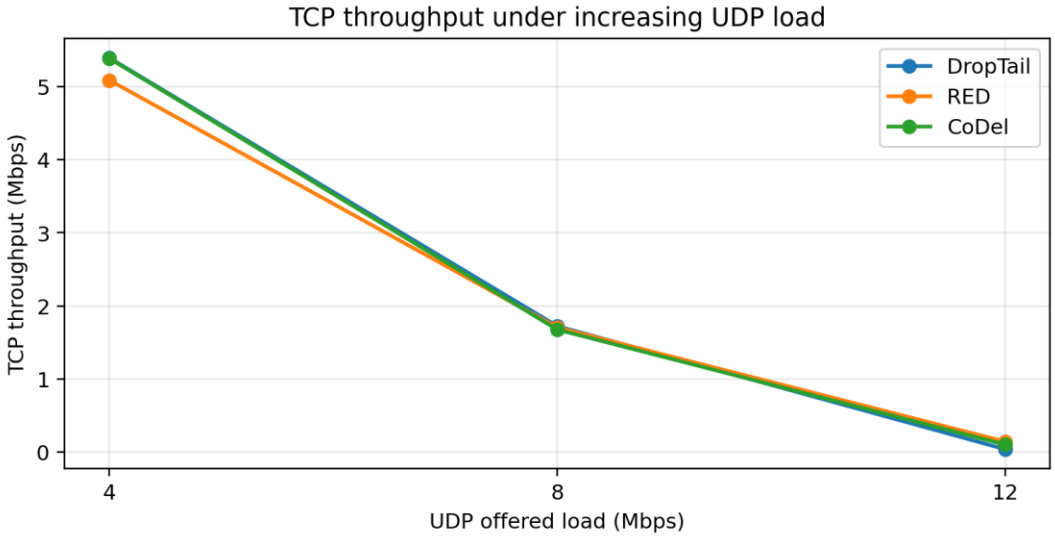


Figure 2. TCP throughput as the UDP offered load increases.

The most pronounced result is the fall in TCP throughput. At 4 Mbps of UDP load, all three queue disciplines deliver slightly above 5 Mbps of TCP throughput. When the UDP offered rate is raised to 8 Mbps, TCP throughput drops to around 1.7 Mbps. At 12 Mbps, the TCP values become extremely small: 0.041 Mbps for DropTail, 0.145 Mbps for RED, and 0.105 Mbps for CoDel.

This pattern indicates that the UDP source increasingly occupies the capacity of the shared bottleneck. Unlike TCP, the configured UDP source continues transmitting at its selected rate; it does not use TCP congestion control to reduce its offered load when congestion develops. The resulting competition is therefore strongly unfavourable to TCP in the highest-load case.

7.2 UDP throughput

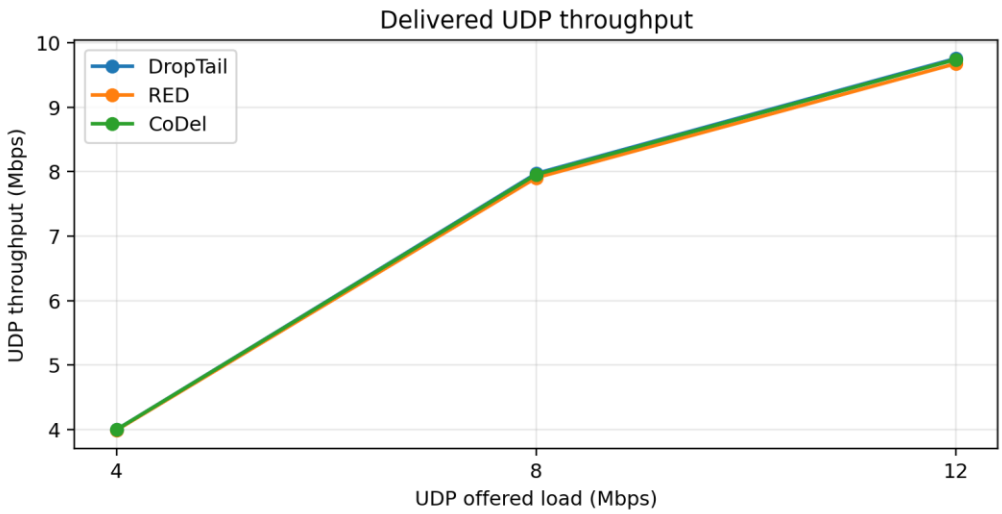


Figure 3. Delivered UDP throughput for the three queue disciplines.

At 4 Mbps and 8 Mbps, the received UDP throughput remains close to the configured offered rate. At 12 Mbps, however, the delivered rate is approximately 9.7 Mbps for every algorithm, below the 12 Mbps offered load. This is consistent with the 10 Mbps bottleneck being unable to carry the full offered UDP traffic alongside the competing TCP traffic.

DropTail records the highest UDP throughput in all three tested cases: 3.998 Mbps, 7.971 Mbps, and 9.756 Mbps. The differences are small in absolute terms, and this result alone does not establish that DropTail provides the best overall performance. Its delay results are notably worse.

7.3 Average delay

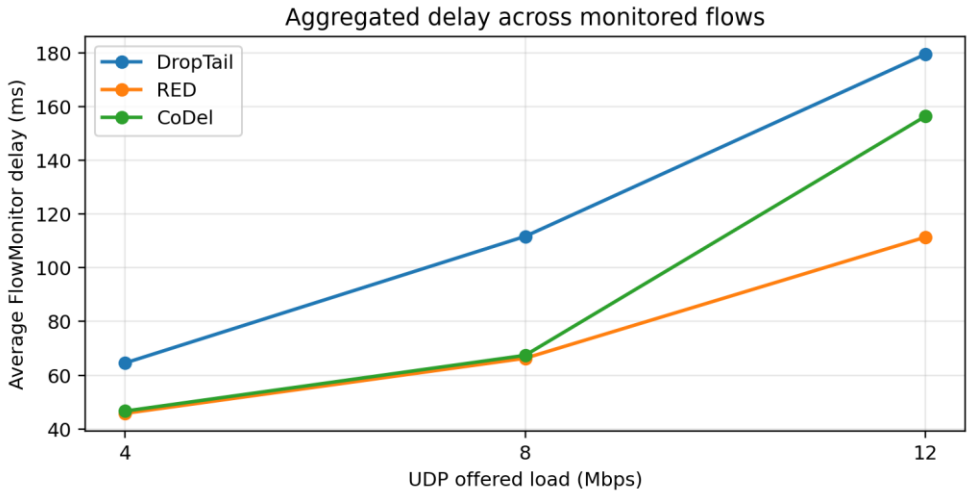


Figure 4. Average FlowMonitor delay aggregated across monitored flows.

Delay rises as the UDP offered load increases. The effect is especially clear with DropTail, where the reported average increases from 64.565 ms at 4 Mbps to 179.381 ms at 12 Mbps. This is consistent with a finite FIFO queue accumulating packets as the offered traffic presses against the bottleneck's service capacity.

RED records the lowest aggregated delay at each tested load: 45.822 ms, 66.312 ms, and 111.343 ms. CoDel is close to RED at the lowest load, with 46.668 ms, but its reported delay reaches 156.298 ms at 12 Mbps. Within this particular simulation and its measurement method, RED gives the lowest average FlowMonitor delay.

7.4 Average jitter

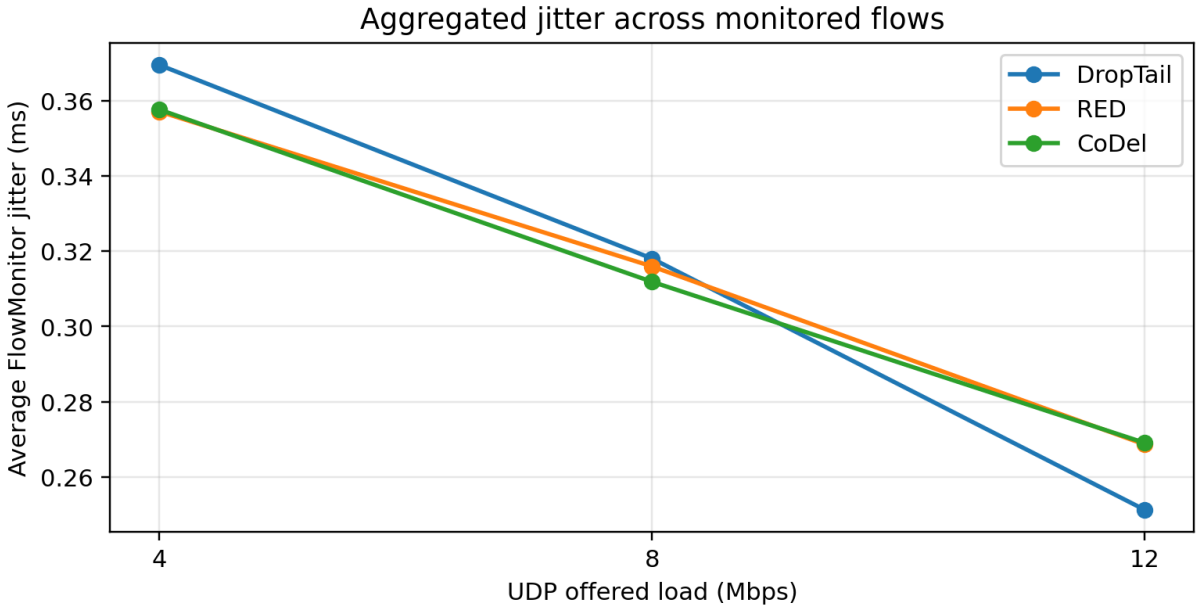


Figure 5. Average FlowMonitor jitter aggregated across monitored flows.

The reported jitter values are close to one another. They range from 0.357–0.369 ms at 4 Mbps and from 0.251–0.269 ms at 12 Mbps. The numerical decrease at higher load should not be interpreted as proof that congestion improves timing stability.

The implementation combines statistics from all monitored flows, including TCP acknowledgement traffic. As congestion changes the packet records and the mix of received flows, the aggregate can move in ways that do not directly describe the jitter experienced by an individual application. Separate TCP-data and UDP-data jitter measurements would make this comparison more meaningful.

7.5 FlowMonitor lost-packet count

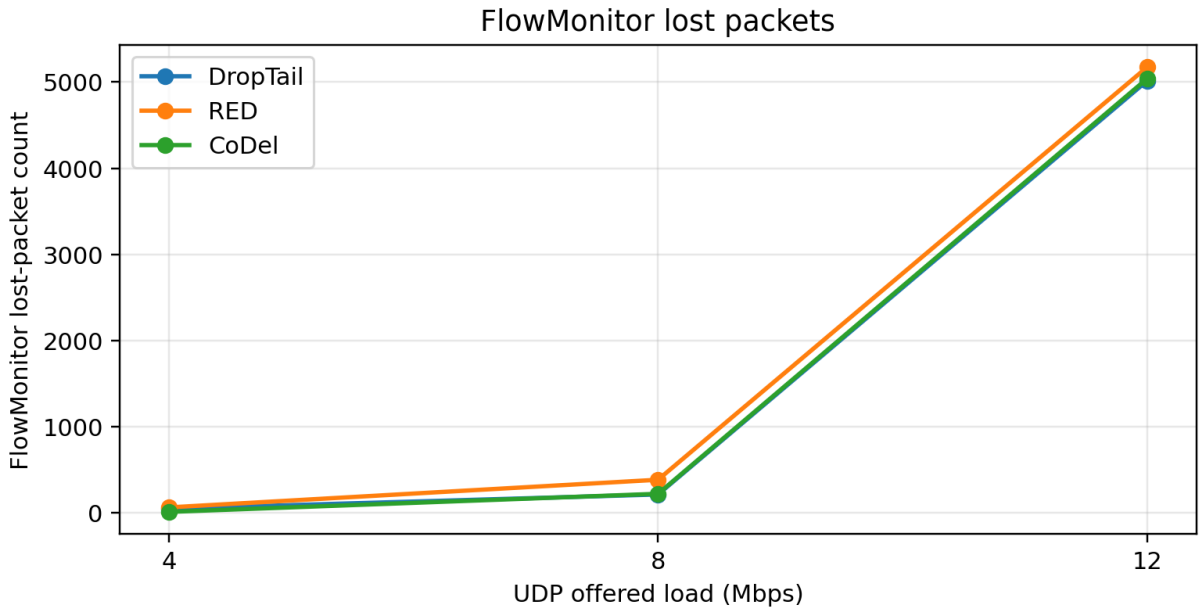


Figure 6. FlowMonitor lost-packet count for each algorithm and UDP load.

At 4 Mbps, CoDel reports 13 lost packets, compared with 51 for DropTail and 65 for RED. At 8 Mbps, DropTail has the smallest reported count at 215, narrowly below CoDel at 223; RED reports 386. At 12 Mbps, DropTail again has the lowest count at 5016, followed by CoDel at 5041 and RED at 5174.

The counts rise sharply between 8 Mbps and 12 Mbps for all three disciplines. This is another indication that the highest offered load places the network under severe congestion. However, these values are FlowMonitor's lostPackets statistics. They must not be presented as exact queue-disc drops, because the program does not attach a direct drop trace or read a queue-disc drop counter.

8. Comparative Analysis

UDP offered load	Lowest reported delay	Fewest FlowMonitor losses	Highest TCP throughput	Highest UDP throughput
4 Mbps	RED (45.822 ms)	CoDel (13)	DropTail (5.392 Mbps)	DropTail (3.998 Mbps)
8 Mbps	RED (66.312 ms)	DropTail (215)	DropTail (1.725 Mbps)	DropTail (7.971 Mbps)
12 Mbps	RED (111.343 ms)	DropTail (5016)	RED (0.145 Mbps)	DropTail (9.756 Mbps)

The comparison does not produce a single winner across every metric. RED is the strongest option for the reported average-delay measure in this dataset. It also retains the highest TCP throughput at the most congested UDP load, although that throughput is still only 0.145 Mbps and remains very low.

CoDel's clearest advantage appears in the 4 Mbps case, where it records the fewest FlowMonitor lost packets. That advantage does not continue at the higher offered rates. DropTail, meanwhile, delivers the highest UDP throughput in every scenario and has the lowest reported loss count at 8 Mbps and 12 Mbps. The cost is higher delay, particularly under the heaviest load.

The TCP results deserve particular attention. At 12 Mbps UDP offered load, every queue discipline leaves TCP with a very small throughput share. This suggests that the traffic mix and the fixed-rate UDP source create a serious fairness problem at the bottleneck. AQM can influence queue behaviour, but it does not automatically make a non-responsive UDP sender behave like a congestion-controlled TCP sender.

The appropriate conclusion is therefore conditional. If the primary criterion is the lowest aggregated delay in this experiment, RED performs best. If the focus is the low-load FlowMonitor loss count, CoDel performs best. If the only criterion is delivered UDP throughput, DropTail has the highest recorded values. A practical network choice would require a wider evaluation, especially one that measures per-flow fairness and direct queue behaviour.

9. Limitations and Measurement Considerations

- Aggregated delay and jitter: The program sums FlowMonitor delay and jitter across all monitored flows and divides by the total received-packet count. These values include different flow types and should not be described as TCP-only or UDP-only application delay or jitter.
- FlowMonitor loss versus queue drops: The lostPackets field is a FlowMonitor statistic based on its packet observation and loss-detection process. The source does not collect queue-disc drop traces, so the reported values are not direct counts of packets dropped by DropTail, RED, or CoDel.
- CoDel defaults: The code sets CoDel MaxSize to 100 packets but does not explicitly set target or interval. The exact model defaults depend on the NS-3 version used and should be recorded in a fully reproducible submission.
- Single reported run: The supplied output contains one reported result for each of the nine scenarios. No multi-seed averages, standard deviations, or confidence intervals are available. Small differences between algorithms should therefore be treated cautiously.
- TCP configuration: The source uses the TCP socket factory and BulkSend but does not explicitly specify a TCP variant. The active default TCP configuration is consequently dependent on the NS-3 version and global settings.
- Aggregate loss and delay interpretation: The experiment is useful for comparing the observed behaviour of the complete configured scenario, but it does not isolate every mechanism responsible for each result.

10. Scope for Further Improvement

The current experiment establishes a useful baseline. A stronger follow-up study could make the measurements more diagnostic, rather than relying only on aggregate end-to-end statistics.

- Record the exact NS-3 release, build configuration, random seed, and run number.
- Repeat every scenario across multiple independent random seeds and report mean values, standard deviations, and confidence intervals.
- Collect FlowMonitor statistics separately for TCP data, UDP data, and TCP acknowledgement flows.
- Attach queue-disc trace sources to record enqueue, dequeue, mark, and drop events at the bottleneck.
- Log queue occupancy over time and record packet sojourn-time distributions, especially for the CoDel runs.
- Explicitly set and report CoDel target and interval, and document all relevant RED attributes.

- Test multiple TCP and UDP flows to examine whether the observed unfairness persists as the number of competing senders changes.
- Add offered-load values around the bottleneck capacity to identify the point at which delay, loss, and throughput begin to deteriorate rapidly.

11. Conclusion

This practical evaluated DropTail, RED, and CoDel in an NS-3 dumbbell network carrying concurrent TCP and UDP traffic. The shared inter-router link was configured at 10 Mbps, while the UDP offered load was increased from 4 Mbps to 8 Mbps and then 12 Mbps. The same network and application timing were retained across the nine algorithm/load combinations.

The central finding is that increasing UDP load sharply reduces TCP throughput and increases the reported average delay. At the highest offered load, TCP throughput falls below 0.15 Mbps for all three queue disciplines. The UDP source continues transmitting at a fixed rate, and the results indicate that this traffic can dominate the shared bottleneck.

RED records the lowest aggregated FlowMonitor delay at every tested load. CoDel records the fewest FlowMonitor lost packets in the 4 Mbps case, while DropTail records the highest UDP throughput in all three cases and the fewest reported losses at 8 Mbps and 12 Mbps. DropTail also produces the highest average delay throughout the experiment.

The results do not justify declaring one algorithm universally superior. They instead show that queue-management performance depends on the metric being prioritised, the offered traffic, and the configuration of the bottleneck. The measurements also have limits: aggregate FlowMonitor values cannot replace per-flow statistics or direct queue-disc instrumentation. With those qualifications, the experiment demonstrates how congestion management changes the behaviour of mixed traffic and why throughput, delay, jitter, and loss must be considered together.

12. References

1. NS-3 Project, “ns-3 Network Simulator Documentation and Model Library.” https://www.nsnam.org/documentation/
2. S. Floyd and V. Jacobson, “Random Early Detection Gateways for Congestion Avoidance,” IEEE/ACM Transactions on Networking, vol. 1, no. 4, 1993, pp. 397–413.
3. K. Nichols and V. Jacobson, “Controlling Queue Delay,” Communications of the ACM, vol. 55, no. 7, 2012, pp. 42–50.
4. NS-3 Project, “FlowMonitor.” https://www.nsnam.org/docs/models/html/flow-monitor.html